



Cloud para Devs: fundamentos sem enrolação

Subtítulo: Como sair do deploy no susto para um sistema previsível, observável e escalável

Público: dev iniciante/intermediário que já codou app web/API

Editora: Sharebook

Sumário

1. Por que cloud parece mais difícil do que é
 2. O mapa mental mínimo de cloud
 3. Compute: onde seu código realmente roda
 4. Dados e estado: o coração da arquitetura
 5. Rede e segurança sem misticismo
 6. Deploy sem drama: CI/CD e ambientes
 7. Observabilidade: enxergar antes de quebrar
 8. Escala e custo: performance com responsabilidade
 9. Arquitetura de referência para produto real
 10. Plano de 30 dias para virar dev cloud confiável
-

1) Por que cloud parece mais difícil do que é

Se você já trabalhou com software, cloud não é um universo alienígena. É o mesmo jogo de sempre, só com mais peças e mais responsabilidade operacional.

O que assusta no começo:

- muitos serviços com nomes diferentes

- dashboard gigante com 200 botões
- medo de custo inesperado
- medo de “abrir sem querer” uma porta de segurança

A real: você não precisa dominar tudo para entregar valor.

Cloud bem feita é sobre responder cinco perguntas com clareza:

1. Onde meu código roda?

2. Onde meus dados ficam?

3. Quem pode acessar o quê?

4. Como eu publico sem derrubar?

5. Como eu descubro rápido quando algo dá ruim?

Se você souber responder isso de forma consistente, já está na frente de muito time.

O erro clássico de quem começa

Querer aprender por catálogo de serviço.

“Hoje vou estudar 17 serviços da AWS.”

Isso dá sensação de progresso e zero capacidade de decisão.

Aprenda por problema real:

- preciso servir tráfego web

- preciso salvar dados
- preciso autenticar
- preciso deploy contínuo
- preciso monitorar

Cloud não é prova de memorização. É engenharia de trade-off.

2) O mapa mental mínimo de cloud

Antes de discutir ferramenta, grava esse mapa:

- Compute: executa sua aplicação
- Storage/Database: guarda estado
- Network: conecta e controla tráfego
- Identity/Security: define quem pode fazer o quê
- Observability: mede saúde do sistema
- Delivery: publica mudanças com segurança

Todo projeto sério em cloud é combinação desses blocos.

Camadas que importam no dia a dia

1. Produto: regra de negócio

2. Aplicação: API/frontend/jobs

3. Infra: runtime, banco, rede, permissões

4. Operação: deploy, monitoramento, incidentes

Quando dá problema, quase sempre é falha de alinhamento entre camadas.

Exemplo clássico:

- app ótima localmente
- deploy em produção sem variáveis corretas
- app sobe mas quebra na primeira query

Não era bug de negócio. Era bug de operação.

Regra prática

Sempre versionar três coisas juntas:

- código
- configuração
- infraestrutura

Sem isso, sua produção vira “artesanato”.

3) Compute: onde seu código realmente roda

Você tem três modelos principais para começar:

1. VM (máquina virtual)

2. Container

3. Serverless

VM

Prós:

- controle total
- fácil de entender para quem veio de servidor tradicional

Contras:

- mais responsabilidade de patch, hardening, automação
- escala geralmente manual no início

Use quando:

- time pequeno
- stack simples
- precisa de previsibilidade e controle direto

Container

Prós:

- empacota runtime + dependências
- reduz “na minha máquina funciona”

- facilita padronização de deploy

Contras:

- ainda exige orquestração/ops
- pode virar complexidade cedo demais se você for direto para Kubernetes sem maturidade

Use quando:

- quer consistência entre ambientes
- tem múltiplos serviços
- precisa escalar com mais organização

Serverless

Prós:

- menos infra para operar
- escala automática por padrão
- ótimo para workloads event-driven

Contras:

- lock-in de provedor pode aumentar
- cold starts e limites de execução
- observabilidade às vezes mais chatinha

Use quando:

- eventos, webhooks, automações, jobs
- tráfego variável
- time quer velocidade com menos gestão de servidor

Decisão simples para não travar

- app monolito inicial + poucos acessos: VM ou container simples
- API com crescimento previsível: container
- automações/eventos pontuais: serverless

A pior decisão não é escolher “errado”. É ficar meses sem escolher.

4) Dados e estado: o coração da arquitetura

Compute pode morrer e renascer. Dado não.

Quem trata banco como detalhe paga caro depois.

Tipos de armazenamento

- Relacional (PostgreSQL, MySQL): consistência, transação, modelagem clara
- NoSQL (documento/chave-valor): flexibilidade e escala para certos padrões
- Objeto (S3/GCS/Azure Blob): arquivos, imagens, PDFs, backups

- Cache (Redis): performance e redução de carga em banco primário

Princípios que evitam tragédia

- 1. Backup testado > backup “configurado”**
- 2. Migração versionada sempre**
- 3. Segredo fora do código**
- 4. Privilégio mínimo no acesso ao banco**
- 5. Ambiente de staging parecido com produção**

Sobre migrations

Se sua migration está divergindo do banco real, não esconda warning no startup.

Conserta a origem:

- snapshot
- migration
- provider de runtime/design-time
- estado real do banco

Ignorar warning é adiar incidente.

Estratégia simples de dados para MVP sério

- banco relacional gerenciado
- storage de objeto para arquivos
- backup diário com retenção definida
- restore testado mensalmente

Isso já te coloca em um nível profissional com esforço acessível.

5) Rede e segurança sem misticismo

Segurança em cloud não é comprar ferramenta cara. É remover exposição desnecessária.

Checklist de base

- TLS obrigatório
- portas públicas mínimas
- acesso administrativo restrito por origem
- secrets em vault/env seguro (não no repositório)
- logs de acesso habilitados
- rotação periódica de credenciais críticas

Modelo mental de rede

Pensa em três zonas:

1. Pública: o que realmente precisa internet

2. Privada de aplicação: serviços internos

3. Privada de dados: banco e componentes sensíveis

Banco aberto para internet “porque facilita” é dívida com juros.

IAM (identidade e acesso)

Erros mais comuns:

- usar credencial root para rotina
- permissões amplas (“*” em tudo)
- credencial compartilhada entre pessoas

Prática saudável:

- usuário/role por contexto
- auditoria de acesso
- política menor possível para cada serviço

Segurança como processo

Não existe estado “100% seguro”.

Existe rotina:

- patch
- revisão de permissão

- hardening básico
- resposta a incidente

Consistência vence heroísmo.

6) Deploy sem drama: CI/CD e ambientes

Deploy manual por SSH até funciona. Até o dia que não funciona.

Pipeline mínimo decente

- 1. push no repositório**
- 2. build**
- 3. testes**
- 4. artefato versionado**
- 5. deploy em staging**
- 6. validação**
- 7. deploy em produção**

Ambientes

No mínimo:

- dev/local

- staging
- prod

Se staging não se parece com prod, staging vira teatro.

Estratégias de deploy

- Rolling: atualiza aos poucos
- Blue/Green: ambiente novo pronto antes de virar tráfego
- Canary: pequena fatia de usuários primeiro

Para time pequeno, rolling com rollback claro já resolve muito.

Regra de ouro

Rollback precisa ser parte do plano, não improvisado pós-caos.

Pergunta obrigatória antes de deploy:

> Se quebrar, volto em quanto tempo e como?

Se a resposta for “a gente vê na hora”, você não está pronto.

7) Observabilidade: enxergar antes de quebrar

Sem observabilidade, produção vira adivinhação.

Você precisa de três pilares:

- Logs (o que aconteceu)
- Métricas (como está o sistema)
- Traces (onde está lento/quebrando no fluxo)

Logs que prestam

- estruturados (json quando possível)
- com correlação por request
- sem vaziar dado sensível
- retenção definida

Métricas essenciais para começar

- latência p95/p99
- taxa de erro
- throughput
- uso de CPU/memória
- conexões de banco

Alertas úteis

Alerta bom é acionável.

Ruim:

- “CPU > 50% por 1 min” sem contexto

Bom:

- “erro 5xx > 3% por 5 min no endpoint crítico”

SLO/SLI na prática

Não precisa virar Google no dia 1.

Começa simples:

- SLI: % de requests com sucesso
- SLO: 99,5% mês

Com isso, você alinha produto e engenharia sobre qualidade.

8) Escala e custo: performance com responsabilidade

Escalar sem olhar custo é fácil. Difícil é escalar com margem saudável.

5 vazamentos comuns de custo

- 1. ambiente esquecido ligado**
- 2. storage sem lifecycle**
- 3. logs infinitos sem retenção**
- 4. overprovision de banco/compute**
- 5. tráfego de saída ignorado**

FinOps básico para time pequeno

- tag por serviço/projeto
- orçamento com alerta
- revisão semanal de custo
- right-sizing mensal

Escala inteligente

Escalar vertical (máquina maior) é rápido no começo.

Escalar horizontal (mais instâncias) exige mais maturidade, mas melhora resiliência.

A ordem normal:

- 1. corrigir gargalo óbvio**
- 2. cache onde faz sentido**
- 3. otimizar query**
- 4. só então adicionar mais infra**

Hardware não corrige arquitetura ruim para sempre.

9) Arquitetura de referência para produto real

Vamos desenhar uma arquitetura pragmática para um app de catálogo + download de ebook:

Componentes

- frontend web
- API backend
- banco relacional gerenciado
- storage de objetos para PDFs/imagens
- cache para leitura frequente
- fila para tarefas assíncronas (email, processamento)
- observabilidade centralizada

Fluxo simplificado

- 1. usuário acessa frontend**
- 2. frontend chama API**
- 3. API lê banco/cache**
- 4. download usa URL controlada para storage**
- 5. eventos importantes vão para fila**
- 6. workers processam assíncrono**

Segurança nesse desenho

- API atrás de TLS
- banco sem exposição pública
- acesso ao storage por permissão mínima
- audit log de ações administrativas

Escalabilidade nesse desenho

- frontend e API com auto scaling básico
- cache reduz carga de leitura
- workers desacoplam pico de tarefas

Operação nesse desenho

- deploy com pipeline
- rollback simples
- dashboards de latência/erro
- alertas por SLO

Esse setup atende muita operação real sem overengineering.

10) Plano de 30 dias para virar dev cloud confiável

Semana 1 — Base sólida

- subir app simples em ambiente cloud
- configurar domínio + TLS
- armazenar segredo em env/vault
- criar checklist de segurança mínima

Semana 2 — Entrega contínua

- pipeline com build + teste + deploy staging
- deploy em produção com rollback documentado
- separar config por ambiente

Semana 3 — Dados e observabilidade

- backup automático + teste de restore
- logs estruturados
- dashboard com latência, erro e throughput
- alertas para endpoint crítico

Semana 4 — Escala e custo

- revisar gargalos reais
- aplicar cache em ponto de dor
- configurar orçamento e alertas de custo
- documentar arquitetura alvo e próximos passos

Se você executar isso com disciplina, já vira referência no time para operação confiável.

Apêndice A — Checklist de produção (versão curta)

Segurança

- ☐ TLS ativo
- ☐ portas mínimas expostas
- ☐ segredo fora do código
- ☐ IAM por privilégio mínimo
- ☐ logs de acesso habilitados

Deploy

- ☐ pipeline com teste
- ☐ staging funcional
- ☐ rollback testado
- ☐ changelog de release

Dados

- ☐ backup automático

- [] restore testado
- [] migration versionada
- [] acesso ao banco restrito

Observabilidade

- [] logs estruturados
- [] métricas de latência/erro
- [] alertas acionáveis
- [] dashboard de saúde

Custo

- [] tags por serviço
- [] orçamento com alerta
- [] rotina de revisão semanal
- [] lifecycle em storage/logs

Apêndice B — Glossário sem complicação

- IaaS: infraestrutura como serviço (você gerencia mais coisa)
- PaaS: plataforma gerenciada (menos operação manual)

- Serverless: execução sob demanda sem gerenciar servidor diretamente
 - SLA: compromisso de disponibilidade do provedor
 - SLO: meta de confiabilidade do seu serviço
 - SLI: métrica usada para medir o SLO
 - RTO: tempo alvo para voltar após incidente
 - RPO: quanto dado você aceita perder em pior cenário
-

Fechamento

Cloud deixa de ser assustadora quando você troca “catálogo de serviços” por “decisões de arquitetura”.

Você não precisa saber tudo.

Precisa saber o suficiente para entregar com segurança, previsibilidade e velocidade.

A mentalidade certa é:

- simplicidade primeiro
- automação onde dói
- observabilidade desde cedo
- qualidade operacional como parte do produto

Se fizer isso, você não vira só um dev que “sobe aplicação”.

Vira um dev que constrói sistemas que aguentam o mundo real.

Fim da versão v1